UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingenieria de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 01
TAD, Estructuras estaticas y Filas Secuenciales

Ciclo 2026-1

TAD Tipo Abstracto de Datos

1. Introducción
2. Definición
3. Formato TAD
4. Representación
   - Objeto Abstracto
   - Invariante
   - Especificación de las Operaciones.
   - Clasificación de las operaciones: Ejemplos
5. Diseño
6. Implementación
7. Referencias

1. Estructura de datos

Abstracciones y Espcificaciones

- **Introduccion:**
  - **Abstraccion**. Eliminar lo irrelevante y obtener solo lo importante.
  - **Abstraccion por especificacion**.Solo necesitsmos saber que va hacer la operación y no como lo va hacer. (Encapsulacion: modulo, funcion, clase)
  - **Abstraccion por parametrizacion**. Un algoritmo, un tipo o una variable se definen a travez de unos parametros (“2 + 3”, “a + b”, sumar (T a, T b): Genericidad)
- **Tipos de abstracciones**
  - **Abstracion de datos:** Tipo Abstracto de Datos (TAD)
  - **Abstraccion funcional**: Subprogramas
  - **Abstarcion de Iteradores**: Iteradores

1. Estructura de datos

Abstraccion de iteradores. Pertmite definir un recorrido abstracto sobre los elementos dem una colección**.**

- Para cada elemento i del array x.
  - Hacer la accion sobre i.
- Para cada elemento i de la lista lx.
  - Hacer la accion sobre i.
- Para cada elemento i del conjunto cx.
  - Hacer la accion sobre i.

1. Estructura de datos

Es una forma particular de *organizar, gestionar y almacenar información* en una computadora para que pueda ser utilizada de manera eficiente. En terminos sencillos son vasijas para almacenar informacion/datos.

Elegir la estructura adecuada es fundamental, ya que afecta directamente la velocidad y el rendimiento de un programa o algoritmo.

Se clasifican en:

Lineales

- Arreglos, Filas
- Listas enlazadas
- Pilas y colas

No lineales

- Arboles
- Grafos

1. Introducción

TAD (Tipo Abstracto de Datos)

Es un *modelo teórico que define un conjunto de datos y las operaciones permitidas sobre ellos* (qué hace), *ocultando los detalles de su implementación técnica* (cómo lo hace).

Componentes:

- Nombre
- Objeto abstracto
- Operaciones con dominio y codominio
- Especificacion de las operaciones
- Implementacion

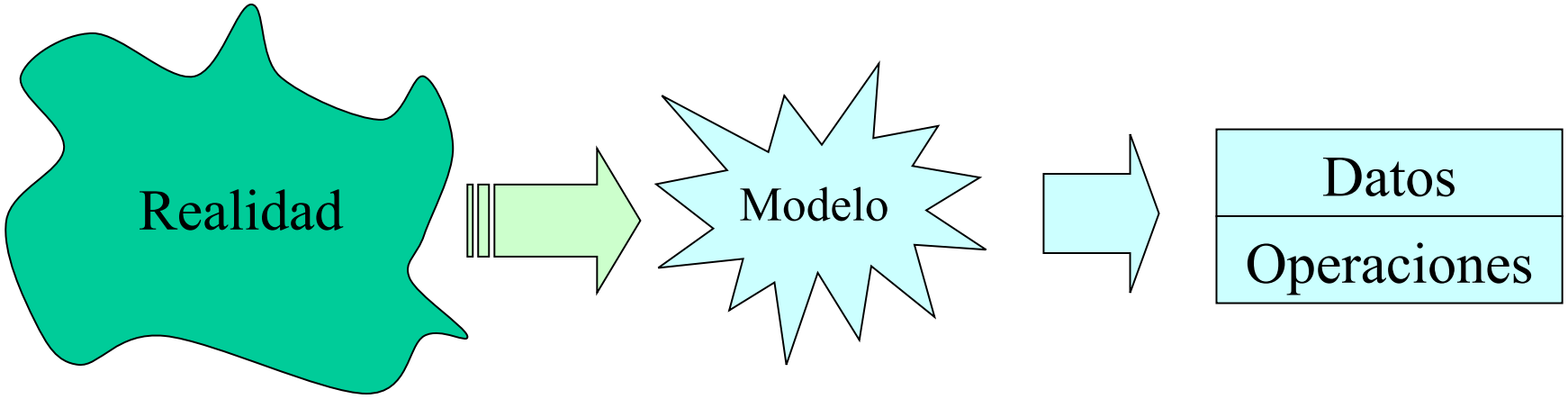
Programas = Algoritmos + datos

Programas = TAD + Interfaz

1. Introducción

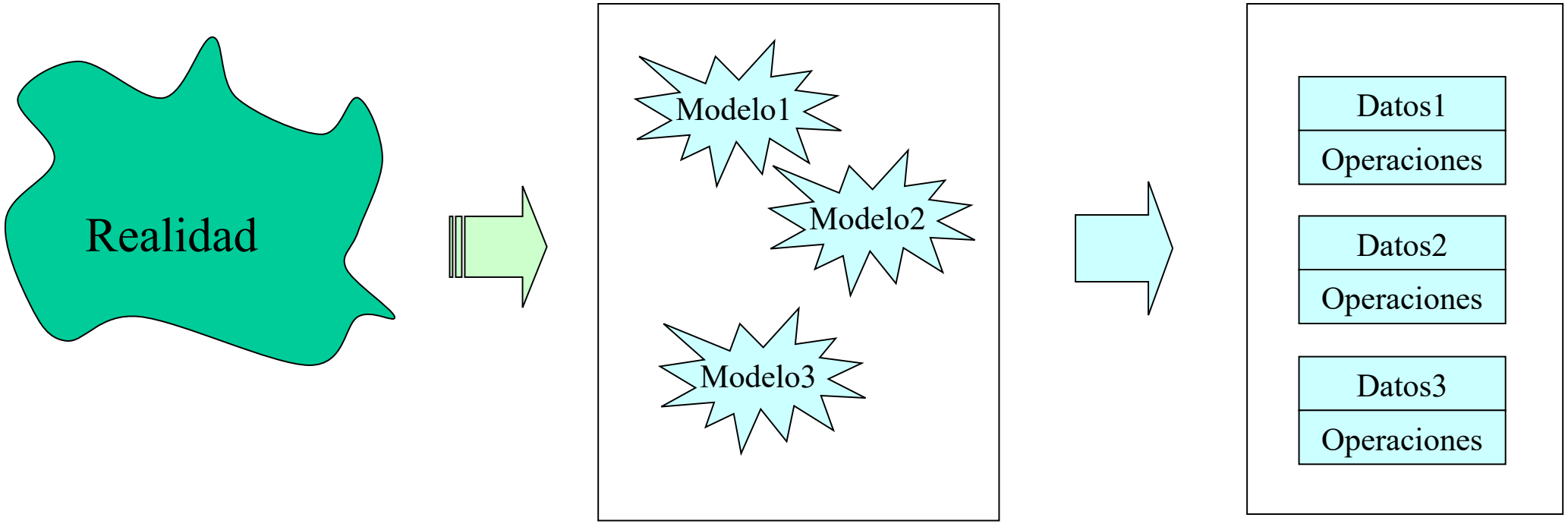
Un TAD es un ***ente cerrado*** y ***autosuficiente***. Esto garantiza portabilidad y reutilización (***encapsulación***)

•Cada elemento de la realidad que participa en el problema tiene un representante en el sw que simule su operación, TAD (datos mas operaciones)



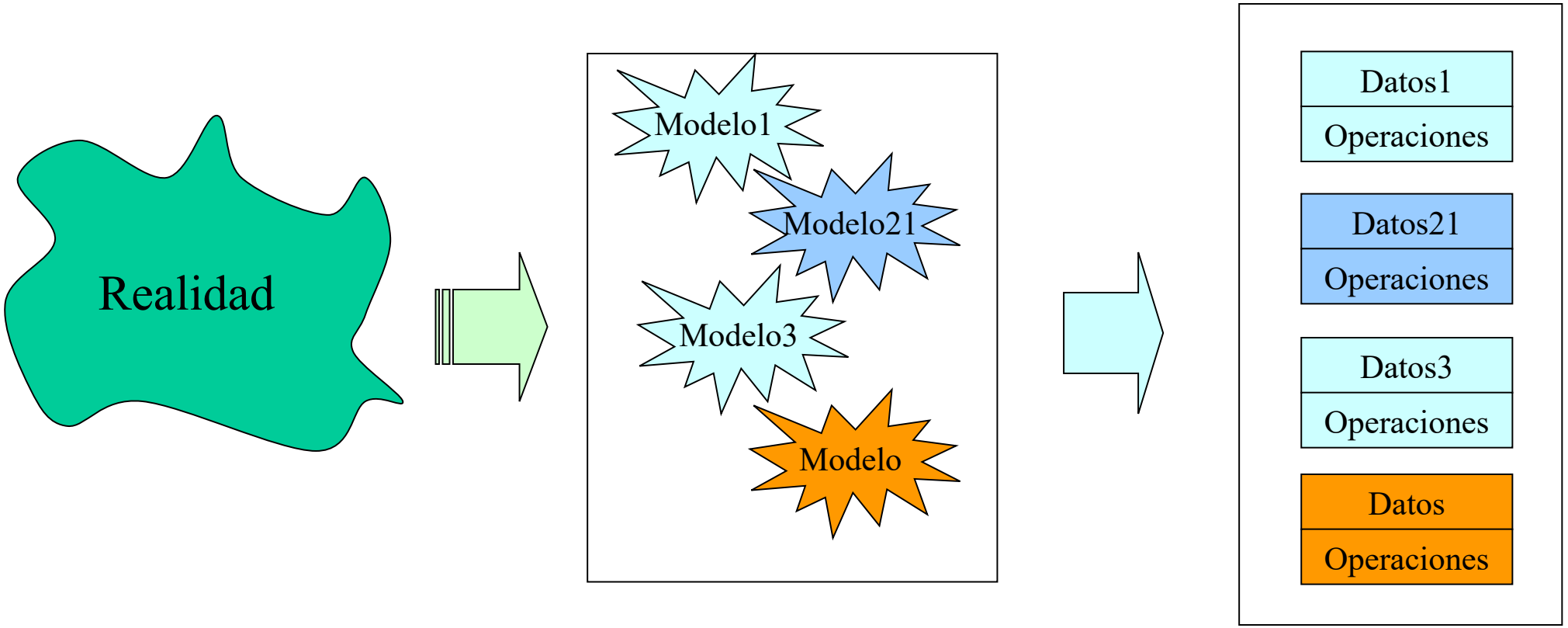
1. Introducción

Un programa está compuesto por un conjunto de TAD cada uno representando un tipo de elemento, estructurados y relacionados igual como lo están en la realidad



1. TAD

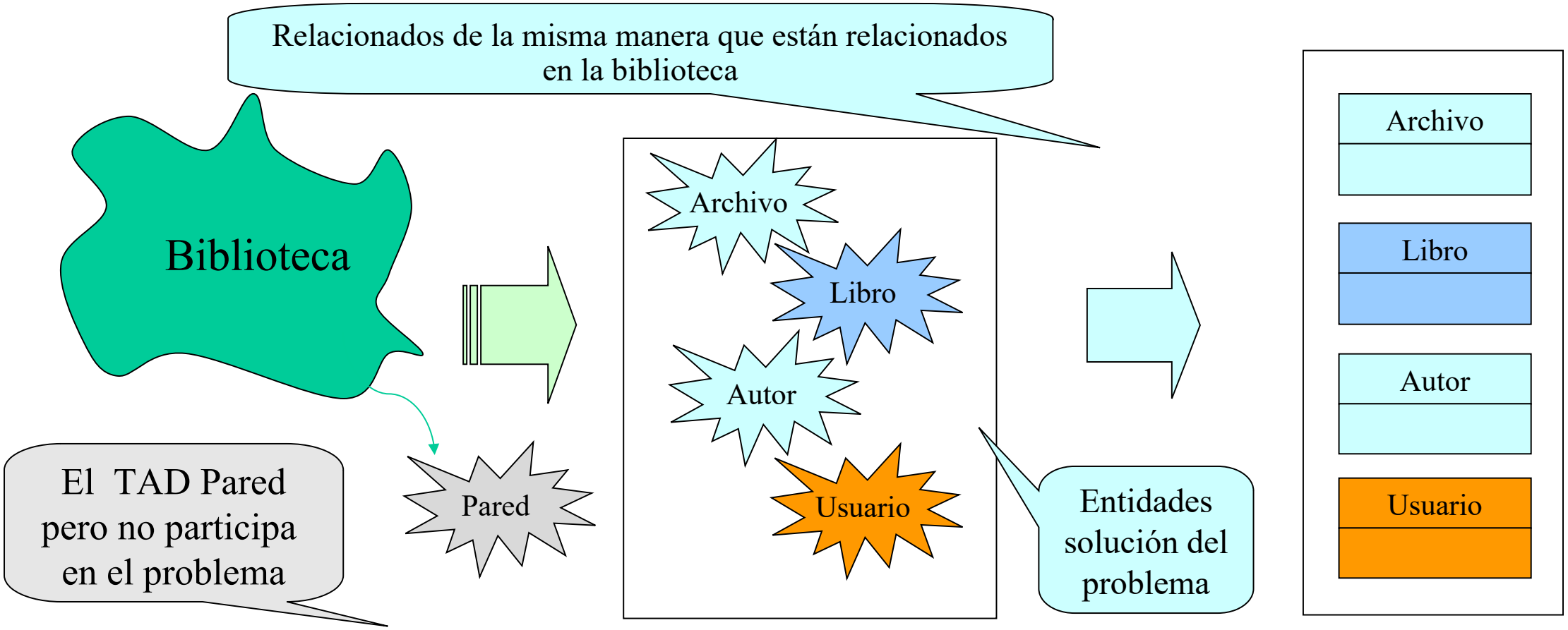
Si la realidad cambia el **software debe evolucionar**



1. TAD Ejemplo

Desarrollar un programa para administrar la información de biblioteca.

En este caso se tiene archivos, usuarios, libros, autores, etc; que participan en el problema, los TAD:



3. Formato del TAD

El TAD se define con un ***nombre***, un *formalismo* para expresar un ***objeto abstracto***, un ***invariante*** y un conjunto de ***operaciones*** sobre este objeto:

TAD < nombre >
< Objeto abstracto >
< Invariante del TAD >
< Operaciones >

4. Representación

***OBJETO ABSTRACTO* :**

El TAD representa a un ***objeto ABSTRACTO*** :

- Es la *representación formal o matemática de los datos que el tipo maneja*, descrita de manera puramente conceptual SIN establecer compromiso con ninguna estructura de datos ni lenguaje de programación.
  - Debe permitir ***expresar las condiciones, relaciones, restricciones y operaciones*** de los elementos modelados sin necesidad de restringirse a una representación interna concreta.
    - Vector : x = {e1, e2, …, en}
    - Conjunto: x = {e1, e2, …, en}
    - Lista: x → {e1 → e2 → … → en → NULL}

5. Invariante

El **OBJETO ABSTRACTO** debe modelar un elemento posible de la realidad.

INVARIANTE. Es la representación formal o matemática de los datos que el tipo maneja, descrita de manera puramente conceptual

- **Permanencia**. Se debe cumplir antes y después de ejecutar cualquier operación del TAD.
- **Restricción:** Define qué estados son legales para la estructura. Si el invariante se rompe, el TAD deja de funcionar correctamente.

Ejemplo:

El TAD conjunto y su notación: $A = \{ x_1, x_2, \ldots, x_n \}$.

El invariante debe exigir que los x_i sean del mismo tipo de dato y que sean diferentes entre si para un objeto abstracto modele realmente un ***Conjunto.***

6. Operaciones, dominio y codominio

Las **operaciones de un TAD** son las *funciones o métodos que permiten interactuar con él*. Cuando las definimos formalmente mediante su **dominio** y **codominio**, estamos estableciendo su "firma" o contrato matemático:

- **Dominio.** Es el conjunto de valores de entrada que la operación recibe. Incluye tanto el propio TAD como cualquier otro dato adicional necesario
- **Codominio**, Es el conjunto de valores que la operación devuelve (el resultado TD, el TAD modificado, booleano).

<operación1>	<dominio>	<codominio>
< ... >	< ... >	< ... >
<operaciónk>	<dominio>	<codominio>

6. Operaciones, dominio y codominio

Clasificacion de las Operaciones

Se dividen en cuatro categorías principales según su relación con el TAD:

- **Constructoras.** Crean una instancia nueva del TAD desde cero.
- **Destructoras.** Eliminan la instancia del TAD.
- **Modificadoras o Transformadoras.** Reciben el TAD, lo modifican y devuelven el TAD actualizado.
- **Analizadoras.** Permiten consultar información del TAD sin modificarlo.

Persistenciaa:

- **Guardar / Serializar:** Traduce el objeto abstracto a un formato que puede almacenarse.
- **Recuperar:** Reconstruye el objeto abstracto a partir de los datos almacenados.

6. Especificación de las Operaciones

La **especificación de operaciones** con pre y postcondicion es la forma de describir **qué hace** una operación de manera lógica y rigurosa, sin escribir código. Define el comportamiento esperado mediante un contrato de "causa y efecto".

- **Precondición** (El "Antes").Es el conjunto de condiciones que **deben ser verdaderas** antes de ejecutar la operación para que esta funcione correctamente.
  - Si la precondición no se cumple, el resultado de la operación es indefinido
- **Postcondición** (El "Después"). Es el efecto resultante despues de la operacion. Es la descripción de cómo cambia el estado del TAD (objeto) y qué resultado devuelve tras ejecutar la operación.
  - Garantiza que si la precondición se cumplió, el objeto abstracto estará en un nuevo estado específico.

6. TAD Vector

TAD Vector TD

Objeto abstracto:

$x = \{e1, e2, \ldots, ei, \ldots, e_{m-1}, e_m\}$

Invariante:

Indices: $i \in [1, m], [0, m>$

Todos los $e_i \in$ TD

Dimension $n <= m$.

Operaciones:

crearArray	True	Array
leerArray	Array × TD × TD	Array
mostrarArray	Array × TD	Secuencia de TD
adicionaDato	Array × TD × TD	Array

6. TAD Vector

Especificacion de las operaciones

ACCION crearArray (Array x, ENTERO dx)
//Crea el Array x vacio
PRE: true
POS: dx = -1

ACCION leerArray (Array x, ENTERO dx)
//Lee o carga n elementos al Array
PRE: x = {} $\wedge$ dx = -1
POS: ”Dimension fuera de Rango” $\vee$ (x = {e1, e2, …, en} dx = [0, n>)

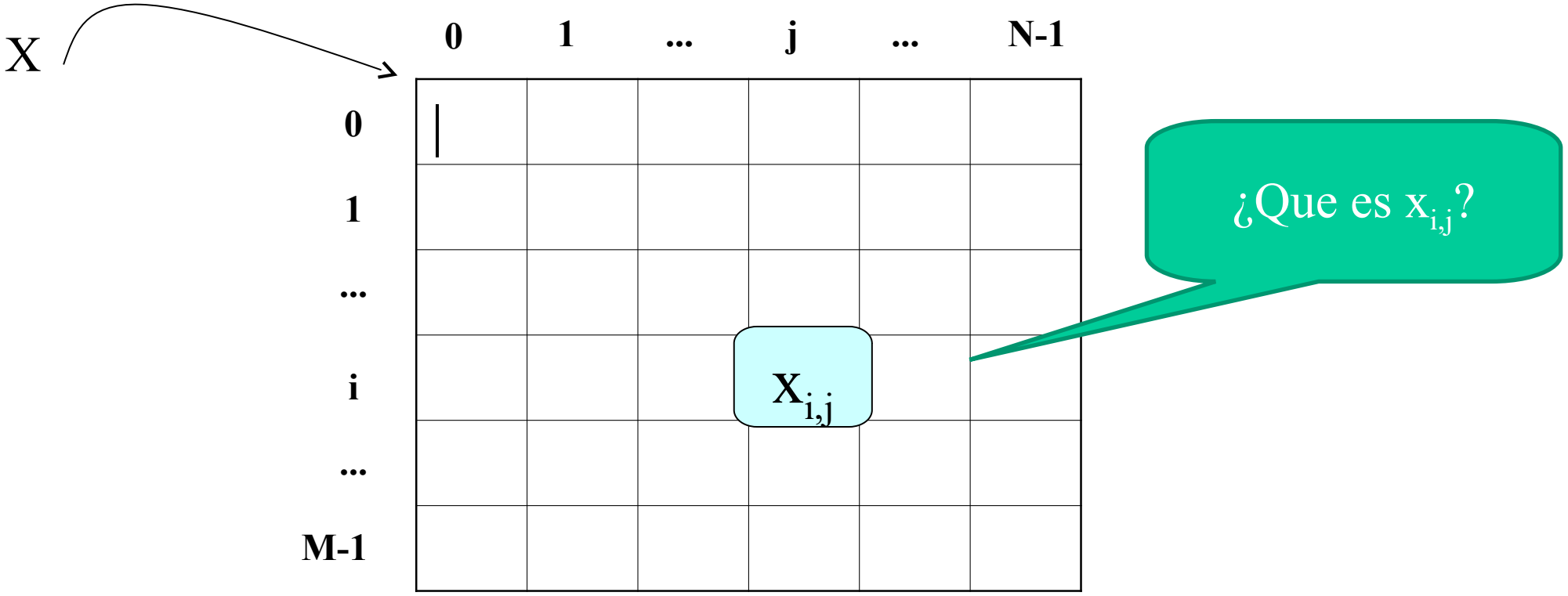
6. TAD Vector

ACCION mostrarArray (Array x, ENTERO dx)
//Muestra los ele,entos del Array x
PRE: (x = {} $\wedge$ dx = -1) $\vee$ (x = {e1, e2, …, en}
POS: ”Array vaciooo” $\vee$ x = {e1, e2, …, en}

ACCION buscarDato (Array x, ENTERO dx, TD dato)
//Busca el elemento dato en el vector x
PRE: (x = {} $\wedge$ dx = -1) $\vee$ (x = {e1, e2, …, en} $\wedge$ dato $\in$ x)
POS: ”Array vaciooo” $\vee$ (dato, “No encontrado” $\vee$ dato, “Encontrado”)

4. Representación: OA Matriz

Es posible hablar de cada uno de los componentes de una matriz X, de sus dimensiones, de la noción de fila y columna, de la relación de vecindad entre sus elementos, etc; sin necesidad de restringirse a una representación interna concreta



4. Reptesentacion: OA Diccionarios

Cada palabra tiene uno mas significados: objeto abstracto se puede representar por:

a) $<elem_1, elem_2, ..., elem_N>$ / $elem_i$ $\begin{cases} \text{Palabra: Cadena} \\ <sig_1, sig_2, ..., sig_N> \end{cases}$

b) $< [parabra_1 \ <elem_{11}, elem_{12}, ..., elem_{1K}>], [palabra_2 <ele \ \ldots>] \ \ldots [parabra_N \ <elem_{N1}, elem_{N2}, ..., elem_{NK}>] >$

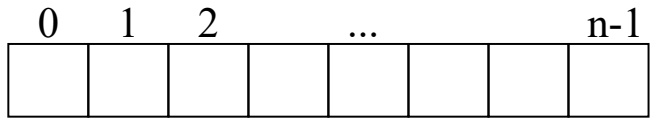
c)

$palabra_1$	$s_{11}, s_{12}, ..., s_{1k}$
$palabra_2$	$s_{21}, s_{22}, ..., s_{2k}$
...	...
...	...
$palabra_N$	$s_{N1}, s_{N2}, ..., s_{Nk}$

Tres formas diferentes, Objeto abstracto. Diccionario

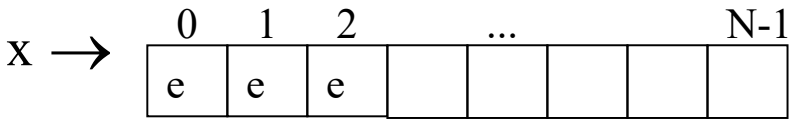
4. Reresentacion: OA Ejemplos

Existen objetos abstractos con una representación gráfica natural:

Conjunto	$\{ x_1, x_2, \ldots, x_n \}$
Cadena de Caracteres	“ $s_1 s_2 \ldots s_n$ ”
Vector	 $x = <e_0, e_1, \ldots, en\text{-}2, en\text{-}1, >$
Polinomio	$c_0 x^0 + c_1 x^1 + c_2 x^2 + \ldots + c_n x^n$
Lista ¿Cual seria el invariante?	$< x_1, x_2, \ldots, x_i, \ldots, x_n >$

5. Ejemplo: TAD Vector de enteros

TAD Vector



{Invariante: $e_i \in Z \wedge N > 0$ }

crearVector	True	Vector
leerVector	**Vector x ENTERO**	**Vector**
mostrarVector	Vector	ENTERO
insertarDatoVector	Vector x ENTERO	Vector
eliminarDatoVector	Vector x ENTERO	Vector
salvarVector	Vector x FILE	f
recuperarVector	…?	…?

ACCION crearVector (Vector x, ENTERO dx)

//Construye y devuelve un vector de dimensión [-1]

PRE: Verdad

POS: dx < 0 o x = { }

5. Ejemplo: TAD Diccionario

TAD Diccionario

$<elem_1, elem_2, ..., elem_N>$ / $elem_i$ { Palabra: Cadena, $<sig_1, sig_2, ..., sig_N>$ }

{Invariante:

$elem_i.palabra < elem_{i+1}.palabra, 1 <= i < N >$ — Palabras ordenadas

$elem_i.sig_r < elem_i.sig_{r+1}, 1 <= i <= N, 1 <= r < k$ — Significados ordenadas

$elem_i = [palabra < sig_1, sig_2, ..., sig_k >], k > 0$ — Toda palabra tiene al menos un significado

}

crearDic		Diccionario
ingresarWordDic	Dicccionario x CadenaxCadena	Diccionario
eliminarWordDic	Dicccionario x Cadena	Diccionario
agregarSignifDic	Dicccionario x CadenaxCadena	Diccionario
numSignifDic	Dicccionario x Cadena	ENTERO
signifDic	Dicccionario x CadenaxENTERO	CADENA

6. Implementación

Para que un TAD funcione es necesario implementarlo con algún lenguaje de programación, significa:

- un tipo de dato creado por el usuario y
- el lenguaje tendrá un NUEVO TIPO DE DATO

6. Implementación

Para implementar un TAD se requieren dos pasos:

Diseñar la ***ESTRUCTURA DE DATOS*** que represente a los ***Objetos Abstractos.***

"Depósito" de los datos

Desarrollar un ***subprograma*** por cada operación del TAD sobre las ***ESTRUCTURAS DE DATOS*** diseñadas.

"Comportamiento" del objeto abstracto en la estructura

6. Implementación

Relación ***Objeto abstracto-Estructura de Datos***: Representar de manera adecuado el ***estado*** de un ***objeto abstracto***

La información debe ser almacenada convenientemente.

Una estructura de datos adecuada

Un esquema de Estructura concreta de Datos: Explica la manera de interpretar y almacenar la información.

Debe verificar que los casos especiales tengan una adecuada representación

6. Implementación

Consideraciones básicas:

- ***Estructura de datos*** capaz de ***almacenar*** y ***recuperar*** toda la información relacionada con el ***objeto abstracto*** a representar. (Empezar por una versión inicial e ir mejorándolo).
- Evaluar y comparar los diseños obtenidos de acuerdo con el problema específico a resolver.

6. Implementación

Estructuras: Declaración de las estructuras para los ejemplos:

```
REGISTRO Conjunto
   ENTERO infe
   ENTERO super
   ENTERO *V
FIN_REGISTRO

Conjunto con
```

Analice

```
REGISTRO NODO
   TD        Val
   NODO   *Sgte
FIN_REGISTRO
```

Se han definido 2 estructuras de TAD conjunto, implemente las operaciones:

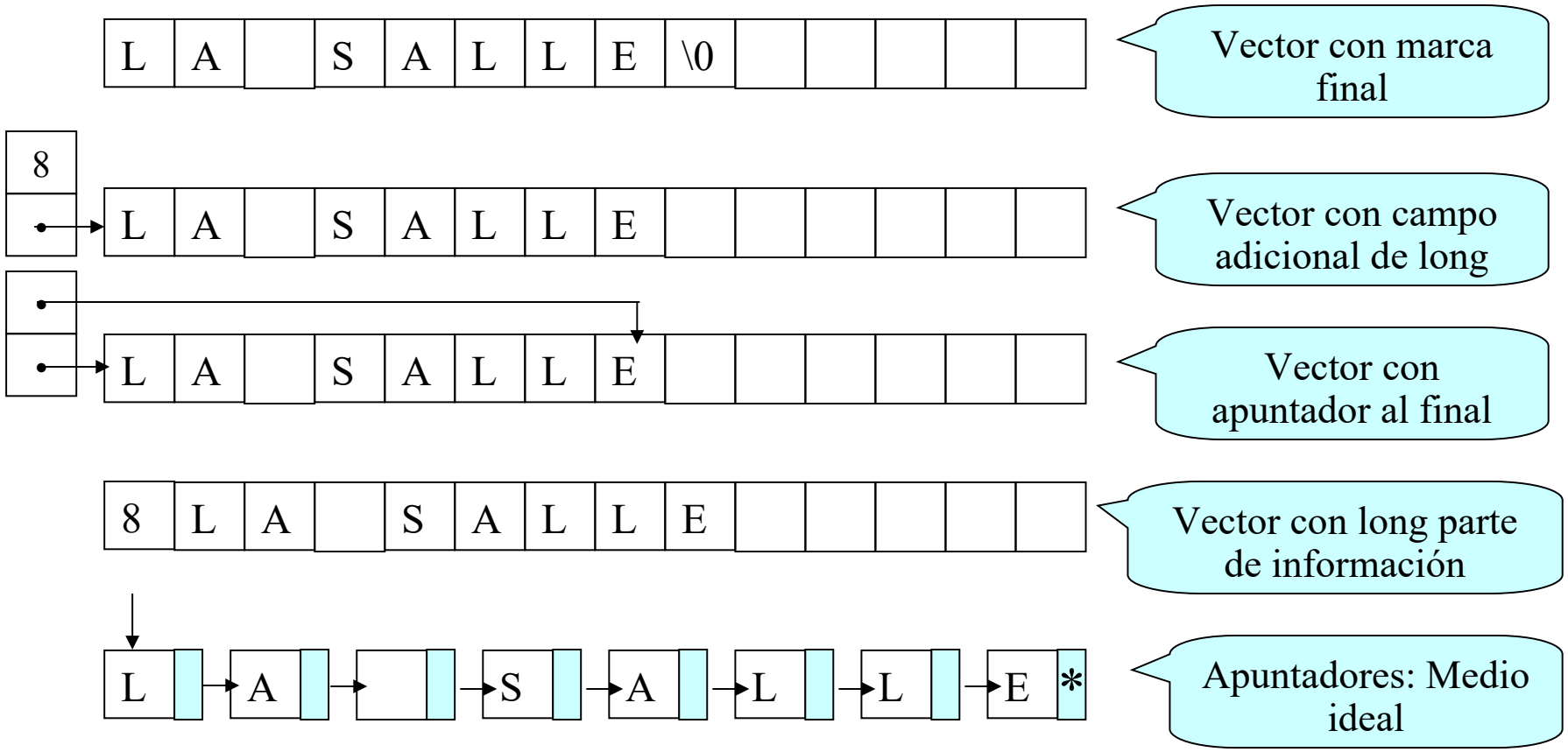
- Crear conjunto
- Leer datos
- Mostrar datos
- Eliminar datos
- Insertar datos
- Operaciones: Union, interseccion, diferencia simetrica
- …

```
REGISTRO Conjunto
   ENTERO infe
   ENTERO super
   NODO *P
FIN_REGISTRO

Conjunto *con
```

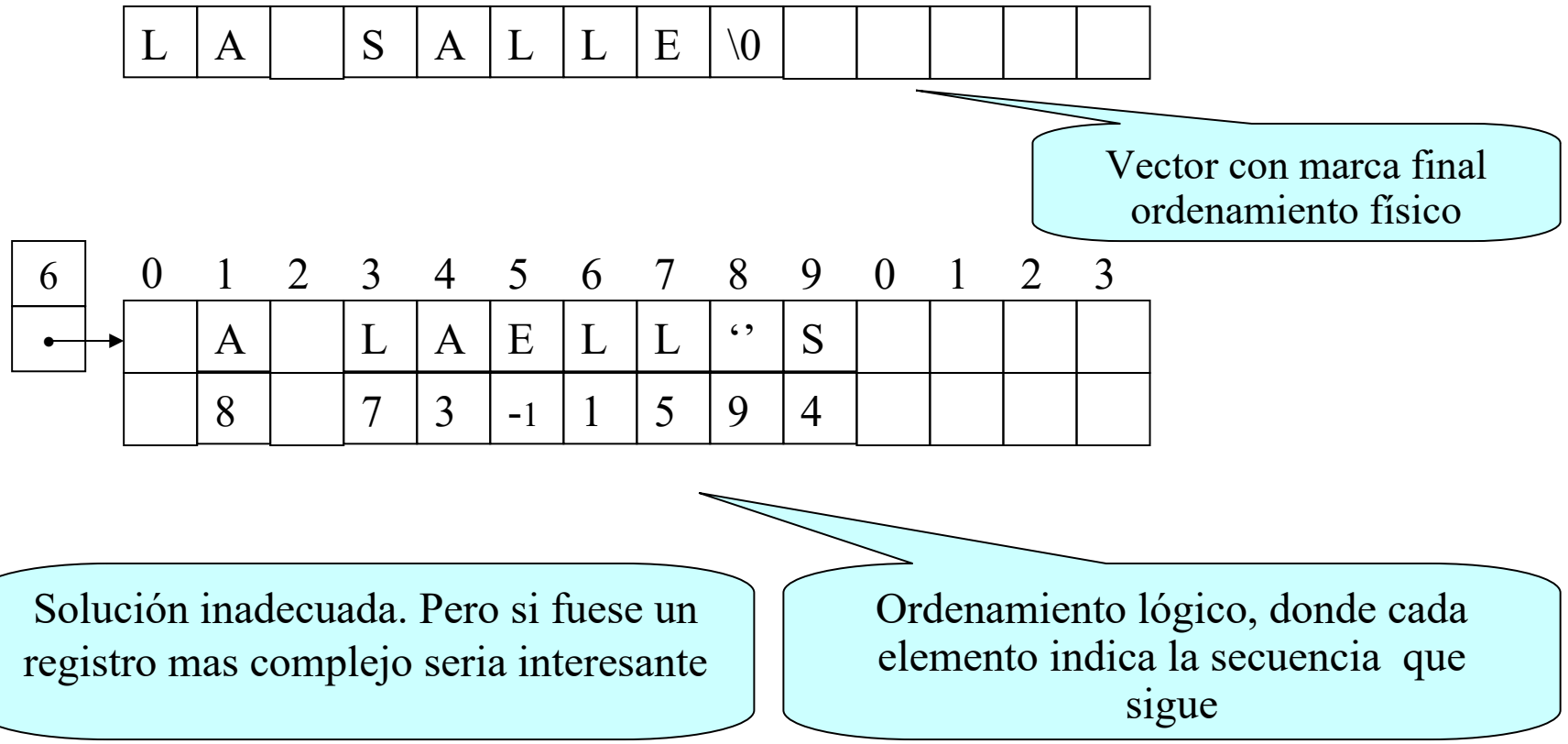
6. Implementación

Representación longitud variable: Atributos que pueden aparecer una o varias veces. Ejemplo Representación de CADENAS



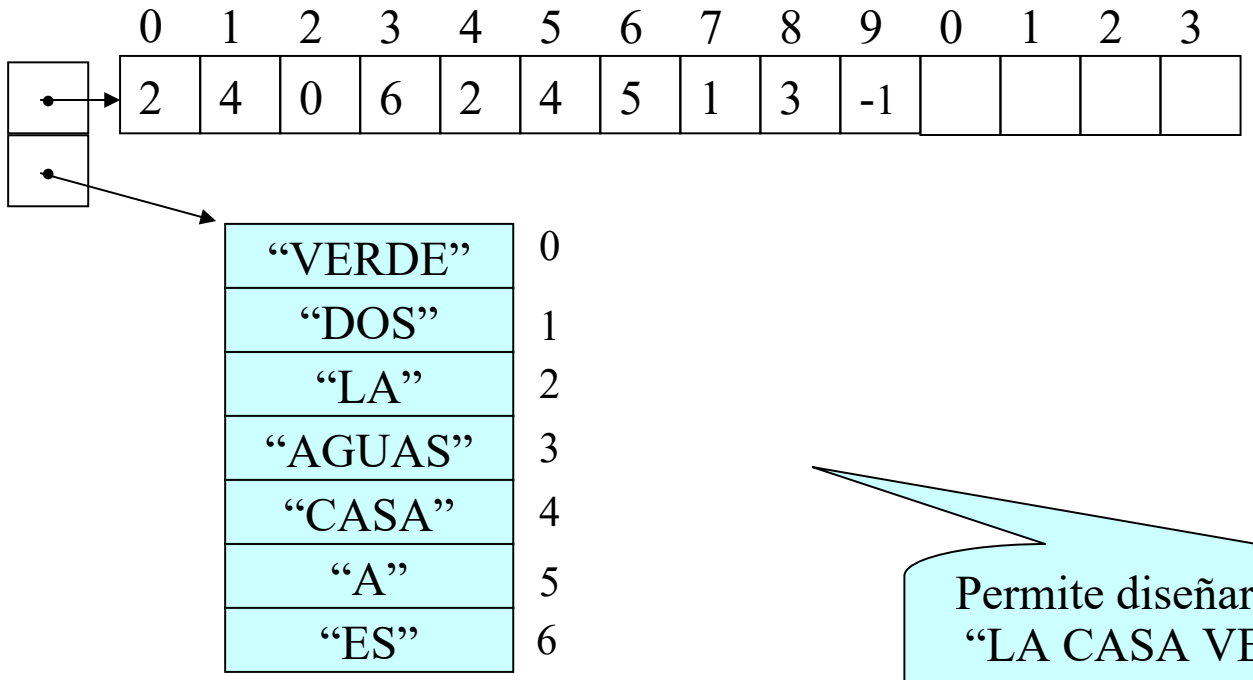
6. Implementación

Ordenamiento Físico vs Lógico: Cuando los componentes deben guardar un orden relativo entre ellos.



6. Implementación

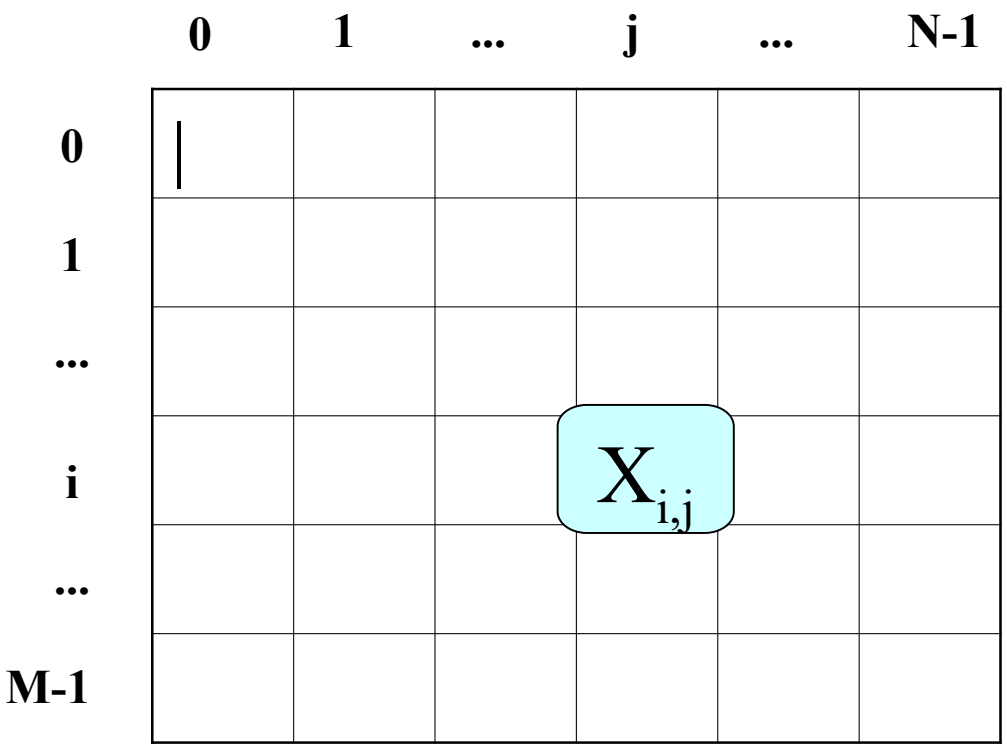
Incorporación de restricciones del Problema: Solo puede aparecer palabras de un conjunto limitado de ellas separadas por blancos



Permite diseñar estructuras compactas: "LA CASA VERDE ES LA CASA A DOS AGUAS"

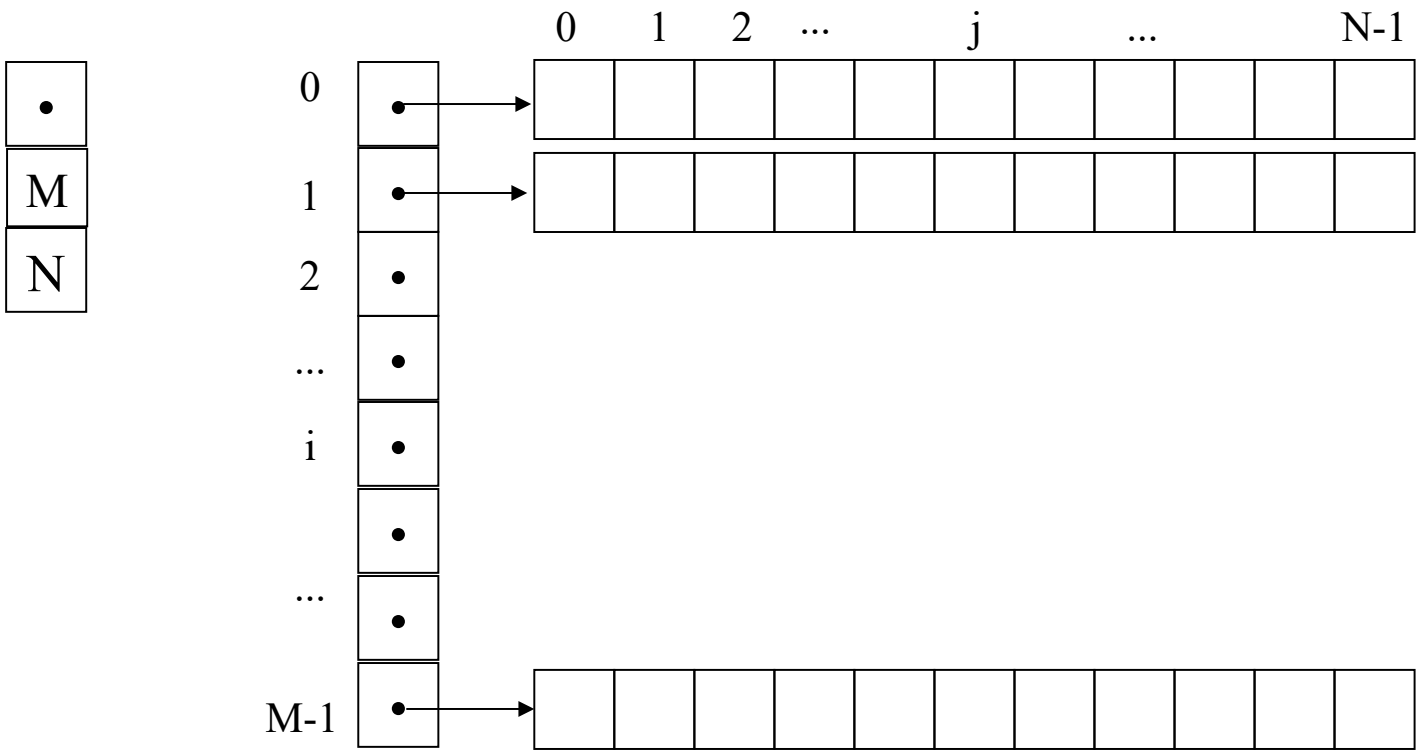
6. Implementación

Estructura de Datos para el TAD Matriz: Se pueden hacer muchos diseños dependiendo del problema



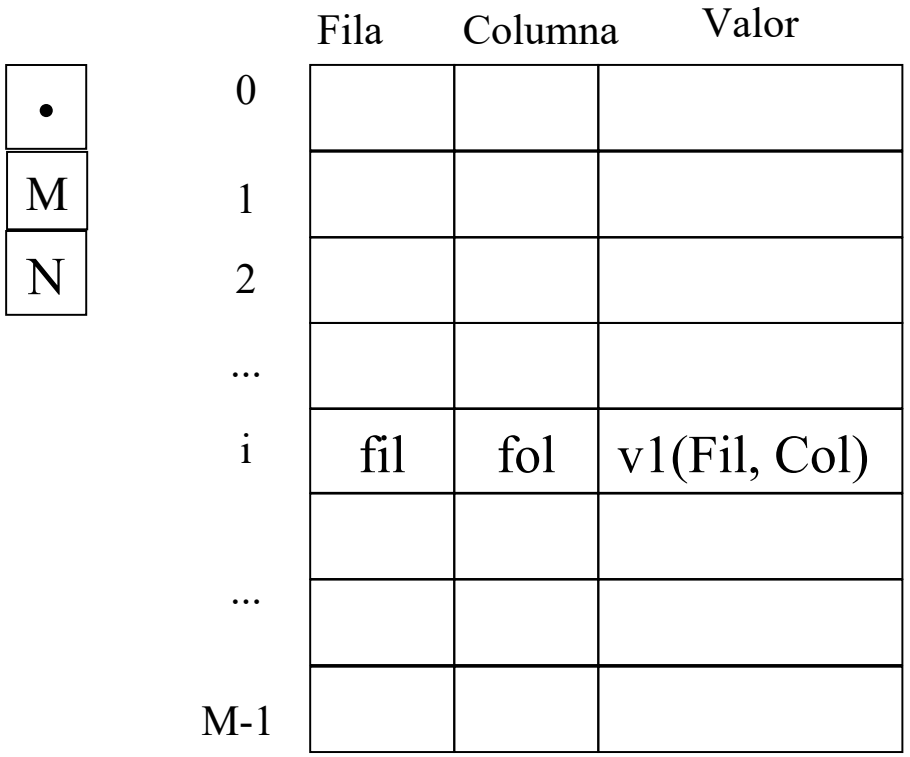
6. Implementación

Estructura de Datos para el TAD Matriz: Se pueden hacer muchos diseños dependiendo del problema **b) Matriz dinámica**



6. Implementación

Estructura de Datos para el TAD Matriz: Se pueden hacer muchos diseños dependiendo del problema **c) Matriz en vector de registros**

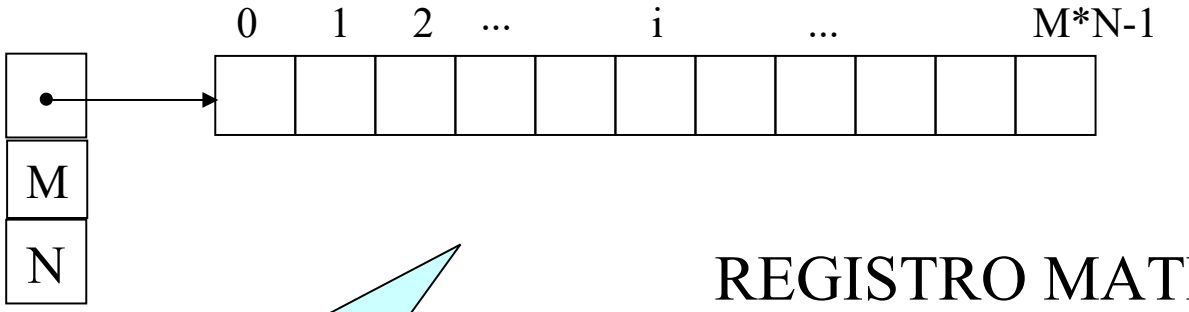



```
REGISTRO MATRIZ
   ENTERO fil
   ENTERO col
   TD        valor
FIN_REGISTRO

MATRIZ  *v, v1[MAX]
```

6. Implementación

Estructura de Datos para el TAD Matriz: Se pueden hacer muchos diseños dependiendo del problema **d) Vector de M*N dinámico**

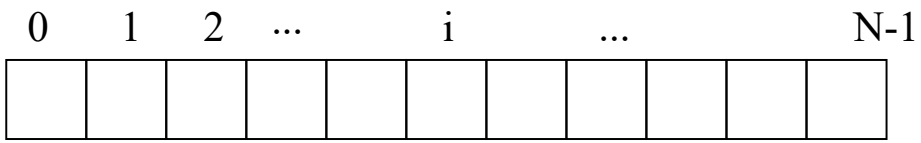


El elemento de la Matriz M[f, c] esta en la casilla v[f*c+i] del vector.

```
REGISTRO MATRIZ
   ENTERO M
   ENTERO N
   TD        *a
FIN_REGISTRO
MATRIZ  *v, v1[MAX]
```

6. Implementación

Estructura de Datos para el TAD Alumnos: Se pueden hacer muchos diseños dependiendo del problema **e) Vector de Registros**



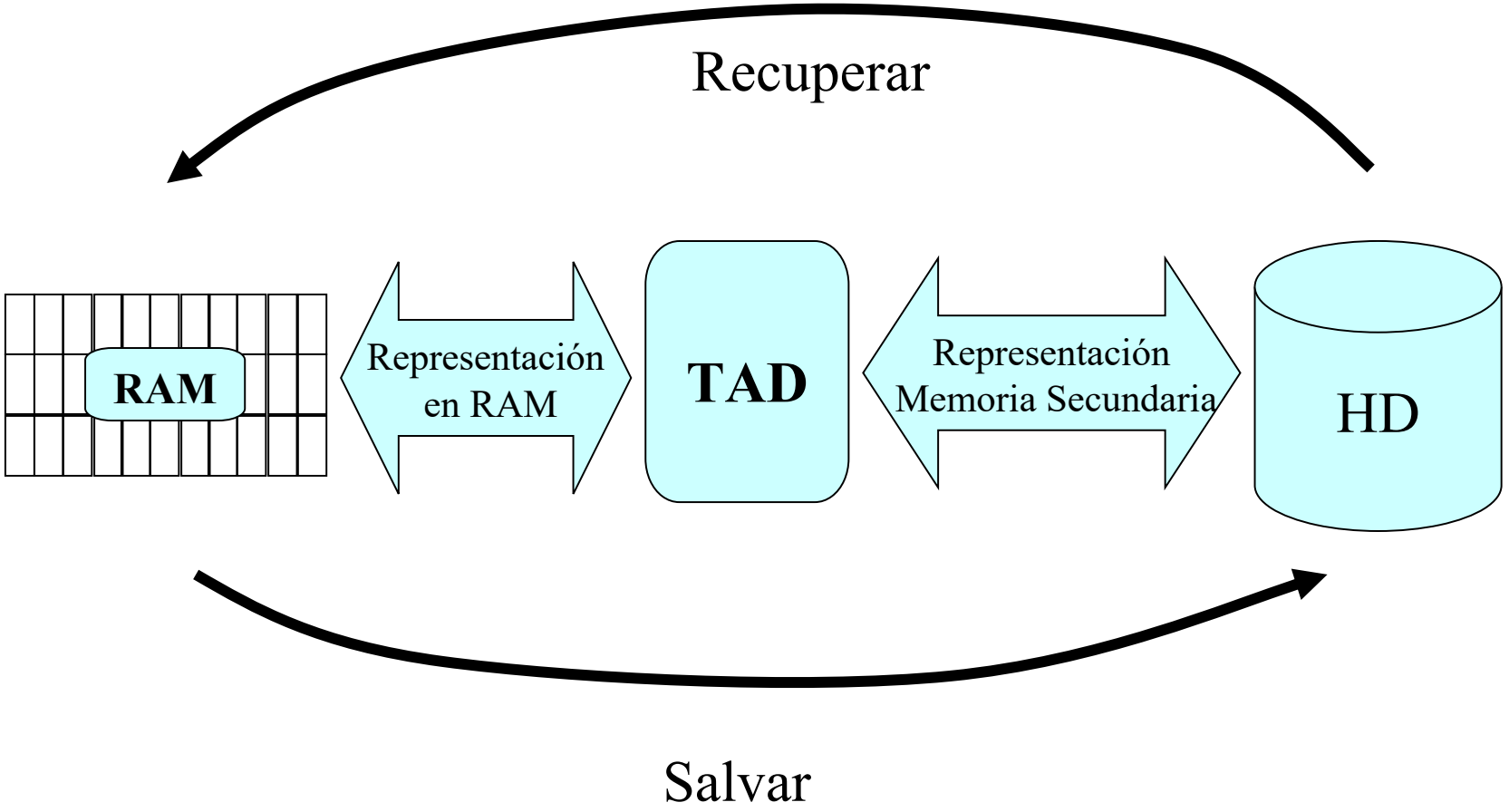
Vector de registros de alumnos.

```
REGISTRO ALUMNOS
   ENTERO      cod
   CARACTER nom[STR80]
   REAL         p[MAXP]
FIN_REGISTRO

ALUMNO  *a, a1[MAX]
```

6. Implementación

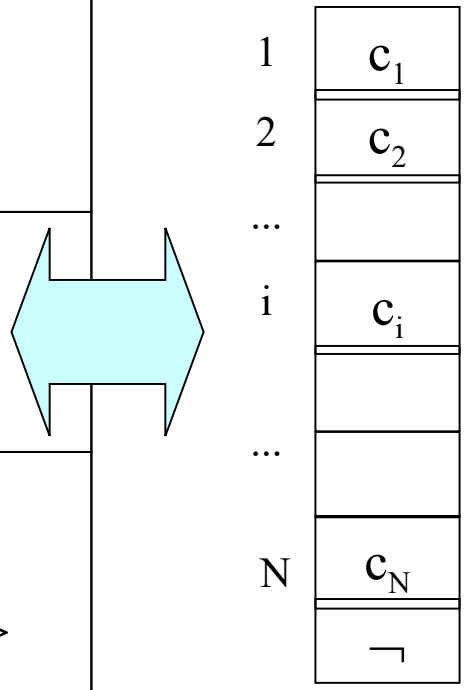
Persistencia de un TAD: Define la manera como se almacena y se recupera de la memoria secundaria (HD) la información asociada a un ***Objeto abstracto.***



6. Implementación

Persistencia: Cada objeto trata de sobrevivir, esto es, la información y solo con esto debe tratarse de reconstruir la representación.

Representación	Formalismo	Esquema Persistencia
c_1 c_2 c_n \0	"$c_1\ c_2\ \ldots c_N$"	
c_1 c_2 c_n	$\{c_1, c_2, \ldots, c_N\}$	
c_1 → c_2 → c_n *	$< c_1, c_2, \ldots, c_N >$	



6. Implementación

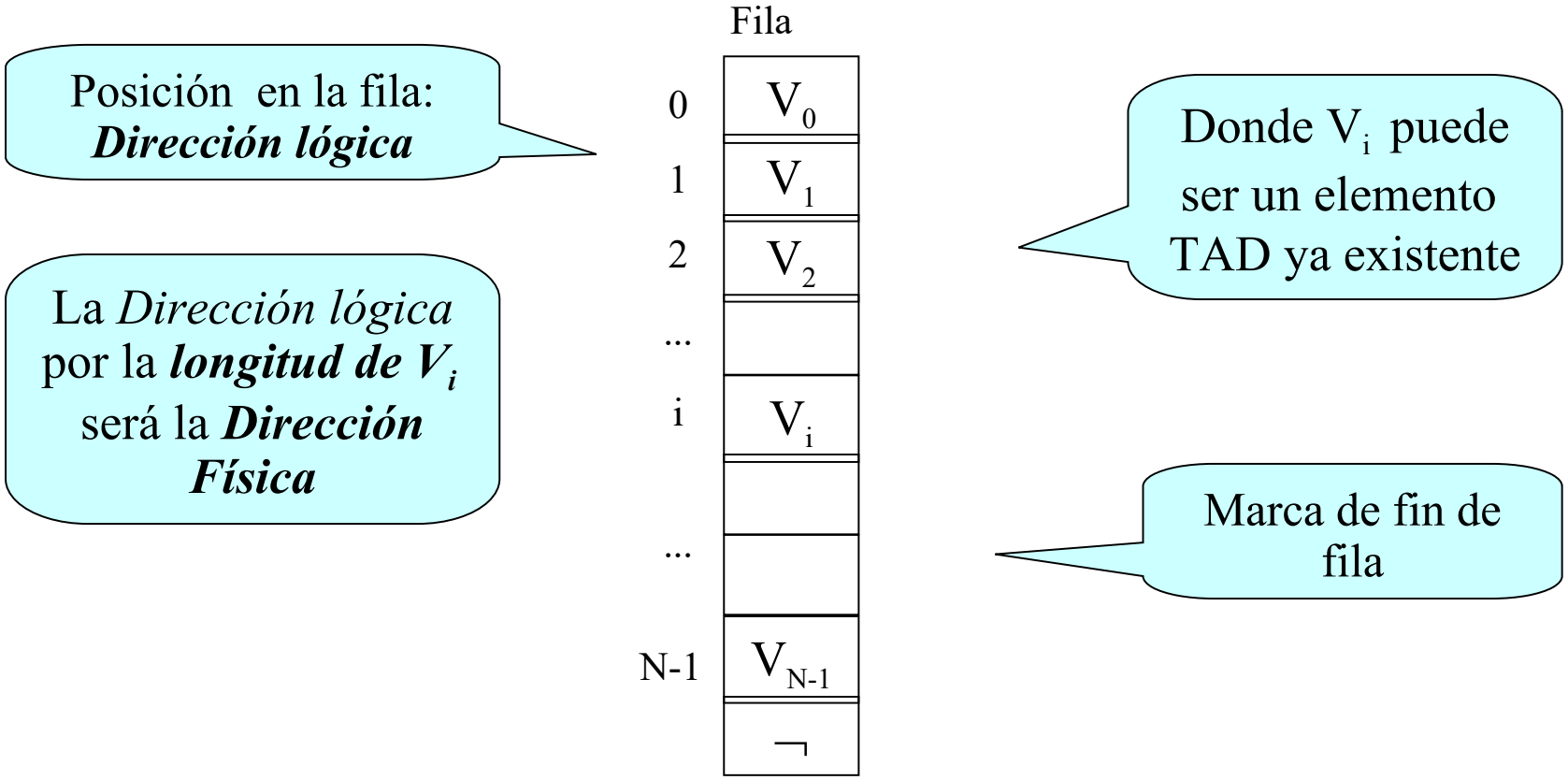
Persistencia de un TAD

Define la manera como se ***SALVA*** y ***RECUPERA*** de memoria secundaria la información asociada con un Objeto Abstracto:

- Cada objeto es responsable de sobrevivir a la ejecución del programa que lo crea.
- Debe ser capaz de representar completamente su estado interno y de implementar operaciones de lectura/escritura de un ***objeto abstracto*** (Administración de memoria secundaria)
- Es conveniente operaciones de persistencia sobre las estructuras de datos en memoria principal.

6. Implementación

Estructura de Datos para el TAD Fila secuencial: Representación en memoria secundaria



Resumen

- TAD permite diseñar objetos independiente al lenguaje y estructura de datos.
- Permite formalizar el diseño y la especificación antes de la implementación.
- Después de diseño y la especificación recién podemos decidir por la estructura adecuada y el lenguaje.
- Para una decisión adecuada de las estructuras requiere de un conocimiento pleno de las operaciones básicas de estas.
- Se reitera el diseño del TAD y su especificación de operaciones es genérico.
- El limite de diseño de TAD solo es la imaginación, creatividad y arte.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesaro en Lima, 19.05.2020, 1820 horas], http://c.conclase.net/ficheros/

Referencias

- Diseno.[Ultima actualización 07.11.2015] [Accesaro en Lima, 18.03.2020, 1820 horas], https://es.wikipedia.org/wiki/Dise%C3%B1o
- Ingenieria.[Ultima actualización 24.02.2014] [Accesaro en Lima, 18.03.2020, 1910 horas], https://es.wikipedia.org/wiki/Ingenier%C3%ADa
- Pensamiento lateral .[Ultima actualización 04.05.2020] [Accesado en Lima, 10.05.2020, 1900 horas]. https://es.wikipedia.org/wiki/Pensamiento_lateral